

6221-21084-ITAI-1371

Group Members:

Edwin Marquez

Kendrick Robinson

Ali Shan

Fernando Reyes

Professor: Sitaram Ayyagari

Date: 10/03/2025

Houston Community College

Houston, Texas

Module 2 Lab Exercise: Tools Used in Machine Learning

Learning Objectives

By the end of this lab, you will be able to:

- Set up and navigate Jupyter Notebook, Google Colab, and VS Code environments
- Install and import essential Python libraries for machine learning
- Create and format professional documentation using Markdown
- Initialize a GitHub repository for your ML projects
- Understand the basic workflow of data science tools

Prerequisites

- Basic understanding of what machine learning is (Module 1)
 - Access to internet for downloading tools and datasets
 - A Google account (for Colab) or local Python installation
-

Part 1: Environment Setup and Tool Overview

What are the main tools we'll use in this course?

Jupyter Notebook/Google Colab: Interactive computing environments where you can write code, see results immediately, and document your work with text and visualizations.

Python Libraries: Pre-written code packages that make machine learning tasks easier:

- **Pandas:** For working with data (like Excel, but more powerful)
- **NumPy:** For mathematical operations on arrays of numbers
- **Matplotlib:** For creating charts and graphs
- **Scikit-learn:** The main library for machine learning algorithms

GitHub: A platform to store, share, and collaborate on code projects

VS Code: A powerful text editor for writing and debugging code

Let's start by setting up our environment!

Environment Setup Instructions

Option 1: Google Colab (Recommended for Beginners)

1. Go to colab.research.google.com
2. Sign in with your Google account
3. Click "New Notebook"
4. You're ready to go! Libraries are pre-installed.

Option 2: Local Jupyter Notebook

1. Install Python from python.org
2. Open terminal/command prompt
3. Run: `pip install jupyter pandas numpy matplotlib scikit-learn`
4. Run: `jupyter notebook`
5. Create a new notebook

Option 3: VS Code

1. Download VS Code from code.visualstudio.com
2. Install Python extension
3. Install Jupyter extension
4. Create a new .ipynb file

For this lab, we recommend starting with Google Colab as it requires no installation.

```
!pip install pandas numpy matplotlib scikit-learn
```

```
Requirement already satisfied: pandas in c:\users\rouge\appdata\local\programs\python\python311\lib\site-packages (2.3.2)
```

```

Requirement already satisfied: numpy in c:\users\rouge\appdata\local\
programs\python\python311\lib\site-packages (2.3.3)
Collecting matplotlib
  Downloading matplotlib-3.10.6-cp311-cp311-win_amd64.whl.metadata (11
kB)
Collecting scikit-learn
  Downloading scikit_learn-1.7.2-cp311-cp311-win_amd64.whl.metadata
(11 kB)
Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\
rouge\appdata\roaming\python\python311\site-packages (from pandas)
(2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in c:\users\rouge\appdata\
local\programs\python\python311\lib\site-packages (from pandas)
(2025.2)
Requirement already satisfied: tzdata>=2022.7 in c:\users\rouge\
appdata\local\programs\python\python311\lib\site-packages (from
pandas) (2025.2)
Collecting contourpy>=1.0.1 (from matplotlib)
  Downloading contourpy-1.3.3-cp311-cp311-win_amd64.whl.metadata (5.5
kB)
Collecting cyclor>=0.10 (from matplotlib)
  Downloading cyclor-0.12.1-py3-none-any.whl.metadata (3.8 kB)
Collecting fonttools>=4.22.0 (from matplotlib)
  Downloading fonttools-4.60.0-cp311-cp311-win_amd64.whl.metadata (113
kB)
----- 0.0/113.8 kB ? eta
-:--:--
----- 10.2/113.8 kB ? eta
-:--:--
----- 71.7/113.8 kB 991.0 kB/s
eta 0:00:01
----- 113.8/113.8 kB 1.1 MB/s
eta 0:00:00
Collecting kiwisolver>=1.3.1 (from matplotlib)
  Downloading kiwisolver-1.4.9-cp311-cp311-win_amd64.whl.metadata (6.4
kB)
Requirement already satisfied: packaging>=20.0 in c:\users\rouge\
appdata\roaming\python\python311\site-packages (from matplotlib)
(25.0)
Collecting pillow>=8 (from matplotlib)
  Downloading pillow-11.3.0-cp311-cp311-win_amd64.whl.metadata (9.2
kB)
Collecting pyparsing>=2.3.1 (from matplotlib)
  Downloading pyparsing-3.2.5-py3-none-any.whl.metadata (5.0 kB)
Collecting scipy>=1.8.0 (from scikit-learn)
  Downloading scipy-1.16.2-cp311-cp311-win_amd64.whl.metadata (60 kB)
----- 0.0/60.8 kB ? eta
-:--:--
----- 60.8/60.8 kB 3.2 MB/s

```

```
eta 0:00:00
Collecting joblib>=1.2.0 (from scikit-learn)
  Downloading joblib-1.5.2-py3-none-any.whl.metadata (5.6 kB)
Collecting threadpoolctl>=3.1.0 (from scikit-learn)
  Downloading threadpoolctl-3.6.0-py3-none-any.whl.metadata (13 kB)
Requirement already satisfied: six>=1.5 in c:\users\rouge\appdata\
roaming\python\python311\site-packages (from python-dateutil>=2.8.2-
>pandas) (1.17.0)
Downloading matplotlib-3.10.6-cp311-cp311-win_amd64.whl (8.1 MB)
----- 0.0/8.1 MB ? eta -:--:--
----- 0.2/8.1 MB 15.0 MB/s eta
0:00:01
----- 1.0/8.1 MB 13.3 MB/s eta
0:00:01
----- 2.5/8.1 MB 20.2 MB/s eta
0:00:01
----- 3.1/8.1 MB 18.2 MB/s eta
0:00:01
----- 4.6/8.1 MB 21.2 MB/s eta
0:00:01
----- 6.1/8.1 MB 23.1 MB/s eta
0:00:01
----- 7.5/8.1 MB 24.1 MB/s eta
0:00:01
----- 8.1/8.1 MB 23.5 MB/s eta
0:00:01
----- 8.1/8.1 MB 22.6 MB/s eta
0:00:00
Downloading scikit_learn-1.7.2-cp311-cp311-win_amd64.whl (8.9 MB)
----- 0.0/8.9 MB ? eta -:--:--
----- 1.4/8.9 MB 43.3 MB/s eta
0:00:01
----- 2.9/8.9 MB 37.3 MB/s eta
0:00:01
----- 4.2/8.9 MB 33.2 MB/s eta
0:00:01
----- 5.2/8.9 MB 30.1 MB/s eta
0:00:01
----- 5.6/8.9 MB 30.1 MB/s eta
0:00:01
----- 7.0/8.9 MB 26.3 MB/s eta
0:00:01
----- 8.3/8.9 MB 26.7 MB/s eta
0:00:01
----- 8.9/8.9 MB 24.7 MB/s eta
0:00:00
Downloading contourpy-1.3.3-cp311-cp311-win_amd64.whl (225 kB)
----- 0.0/225.2 kB ? eta -:--:--
----- 225.2/225.2 kB 14.3 MB/s
```

```
eta 0:00:00
Downloading cycler-0.12.1-py3-none-any.whl (8.3 kB)
Downloading fonttools-4.60.0-cp311-cp311-win_amd64.whl (2.3 MB)
----- 0.0/2.3 MB ? eta -:--:--
----- 1.3/2.3 MB 41.6 MB/s eta
0:00:01
----- 2.3/2.3 MB 29.0 MB/s eta
0:00:01
----- 2.3/2.3 MB 24.1 MB/s eta
0:00:00
Downloading joblib-1.5.2-py3-none-any.whl (308 kB)
----- 0.0/308.4 kB ? eta -:--:--
----- 308.4/308.4 kB 18.6 MB/s
eta 0:00:00
Downloading kiwisolver-1.4.9-cp311-cp311-win_amd64.whl (73 kB)
----- 0.0/73.8 kB ? eta -:--:--
----- 73.8/73.8 kB ? eta 0:00:00
Downloading pillow-11.3.0-cp311-cp311-win_amd64.whl (7.0 MB)
----- 0.0/7.0 MB ? eta -:--:--
----- 1.2/7.0 MB 39.6 MB/s eta
0:00:01
----- 2.7/7.0 MB 34.5 MB/s eta
0:00:01
----- 4.1/7.0 MB 32.6 MB/s eta
0:00:01
----- 5.5/7.0 MB 32.0 MB/s eta
0:00:01
----- 7.0/7.0 MB 31.8 MB/s eta
0:00:01
----- 7.0/7.0 MB 29.8 MB/s eta
0:00:00
Downloading pyparsing-3.2.5-py3-none-any.whl (113 kB)
----- 0.0/113.9 kB ? eta -:--:--
----- 113.9/113.9 kB ? eta
0:00:00
Downloading scipy-1.16.2-cp311-cp311-win_amd64.whl (38.7 MB)
----- 0.0/38.7 MB ? eta -:--:--
----- 1.6/38.7 MB 34.6 MB/s eta
0:00:02
----- 2.9/38.7 MB 30.7 MB/s eta
0:00:02
----- 4.4/38.7 MB 31.3 MB/s eta
0:00:02
----- 5.8/38.7 MB 31.0 MB/s eta
0:00:02
----- 7.3/38.7 MB 31.2 MB/s eta
0:00:02
----- 8.7/38.7 MB 32.5 MB/s eta
0:00:01
```

```
----- 10.0/38.7 MB 30.5 MB/s eta
0:00:01
----- 11.5/38.7 MB 31.2 MB/s eta
0:00:01
----- 12.8/38.7 MB 31.2 MB/s eta
0:00:01
----- 14.3/38.7 MB 29.7 MB/s eta
0:00:01
----- 15.0/38.7 MB 29.8 MB/s eta
0:00:01
----- 16.5/38.7 MB 29.7 MB/s eta
0:00:01
----- 17.9/38.7 MB 27.3 MB/s eta
0:00:01
----- 19.3/38.7 MB 28.5 MB/s eta
0:00:01
----- 20.7/38.7 MB 28.4 MB/s eta
0:00:01
----- 22.2/38.7 MB 29.8 MB/s eta
0:00:01
----- 23.5/38.7 MB 28.5 MB/s eta
0:00:01
----- 25.1/38.7 MB 29.7 MB/s eta
0:00:01
----- 26.4/38.7 MB 31.2 MB/s eta
0:00:01
----- 27.8/38.7 MB 31.2 MB/s eta
0:00:01
----- 29.3/38.7 MB 31.2 MB/s eta
0:00:01
----- 30.6/38.7 MB 29.7 MB/s eta
0:00:01
----- 31.8/38.7 MB 29.7 MB/s eta
0:00:01
----- 33.2/38.7 MB 29.8 MB/s eta
0:00:01
----- 34.7/38.7 MB 29.8 MB/s eta
0:00:01
----- 36.0/38.7 MB 29.7 MB/s eta
0:00:01
----- 37.6/38.7 MB 29.7 MB/s eta
0:00:01
----- 38.7/38.7 MB 28.5 MB/s eta
0:00:01
----- 38.7/38.7 MB 28.5 MB/s eta
0:00:01
----- 38.7/38.7 MB 24.3 MB/s eta
0:00:00
Downloading threadpoolctl-3.6.0-py3-none-any.whl (18 kB)
```

```
Installing collected packages: threadpoolctl, scipy, pyparsing,
pillow, kiwisolver, joblib, fonttools, cyclor, contourpy, scikit-
learn, matplotlib
Successfully installed contourpy-1.3.3 cyclor-0.12.1 fonttools-4.60.0
joblib-1.5.2 kiwisolver-1.4.9 matplotlib-3.10.6 pillow-11.3.0
pyparsing-3.2.5 scikit-learn-1.7.2 scipy-1.16.2 threadpoolctl-3.6.0
```

```
[notice] A new release of pip is available: 24.0 -> 25.2
[notice] To update, run: python.exe -m pip install --upgrade pip
```

```
# Install required libraries (uncomment if needed)
# !pip install pandas numpy matplotlib scikit-learn

# Import libraries with standard aliases
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn import datasets
import warnings
warnings.filterwarnings('ignore') # Hide warning messages for cleaner
output

print("□ All libraries imported successfully!")
print(f"Pandas version: {pd.__version__}")
print(f"NumPy version: {np.__version__}")
```

```
Matplotlib is building the font cache; this may take a moment.
```

```
□ All libraries imported successfully!
Pandas version: 2.3.2
NumPy version: 2.3.3
```

Part 2: Loading and Exploring Your First Dataset

We'll use the famous Iris dataset - a classic dataset for beginners. It contains measurements of iris flowers from three different species.

```
# Load a simple dataset (Iris flowers - a classic beginner dataset)
from sklearn.datasets import load_iris

# Load the data
iris = load_iris()
print("Dataset loaded successfully!")
print(f"Dataset shape: {iris.data.shape}")
print(f"Features: {iris.feature_names}")
print(f"Target classes: {iris.target_names}")

Dataset loaded successfully!
Dataset shape: (150, 4)
```

```
Features: ['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']
Target classes: ['setosa' 'versicolor' 'virginica']
```

```
# Convert to pandas DataFrame for easier handling
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['species'] = iris.target_names[iris.target]
```

```
# Display first few rows
print("First 5 rows of our dataset:")
print(df.head())
```

```
print("\nDataset info:")
print(df.info())
```

First 5 rows of our dataset:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	5.1	3.5	1.4	
1	4.9	3.0	1.4	
2	4.7	3.2	1.3	
3	4.6	3.1	1.5	
4	5.0	3.6	1.4	

	species
0	setosa
1	setosa
2	setosa
3	setosa
4	setosa

Dataset info:

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 150 entries, 0 to 149
```

```
Data columns (total 5 columns):
```

#	Column	Non-Null Count	Dtype
0	sepal length (cm)	150 non-null	float64
1	sepal width (cm)	150 non-null	float64
2	petal length (cm)	150 non-null	float64
3	petal width (cm)	150 non-null	float64
4	species	150 non-null	object

```
dtypes: float64(4), object(1)
```

```
memory usage: 6.0+ KB
```

```
None
```


Part 3: Creating Your First Visualization

Data visualization is crucial in machine learning. Let's create a simple plot to understand our data.

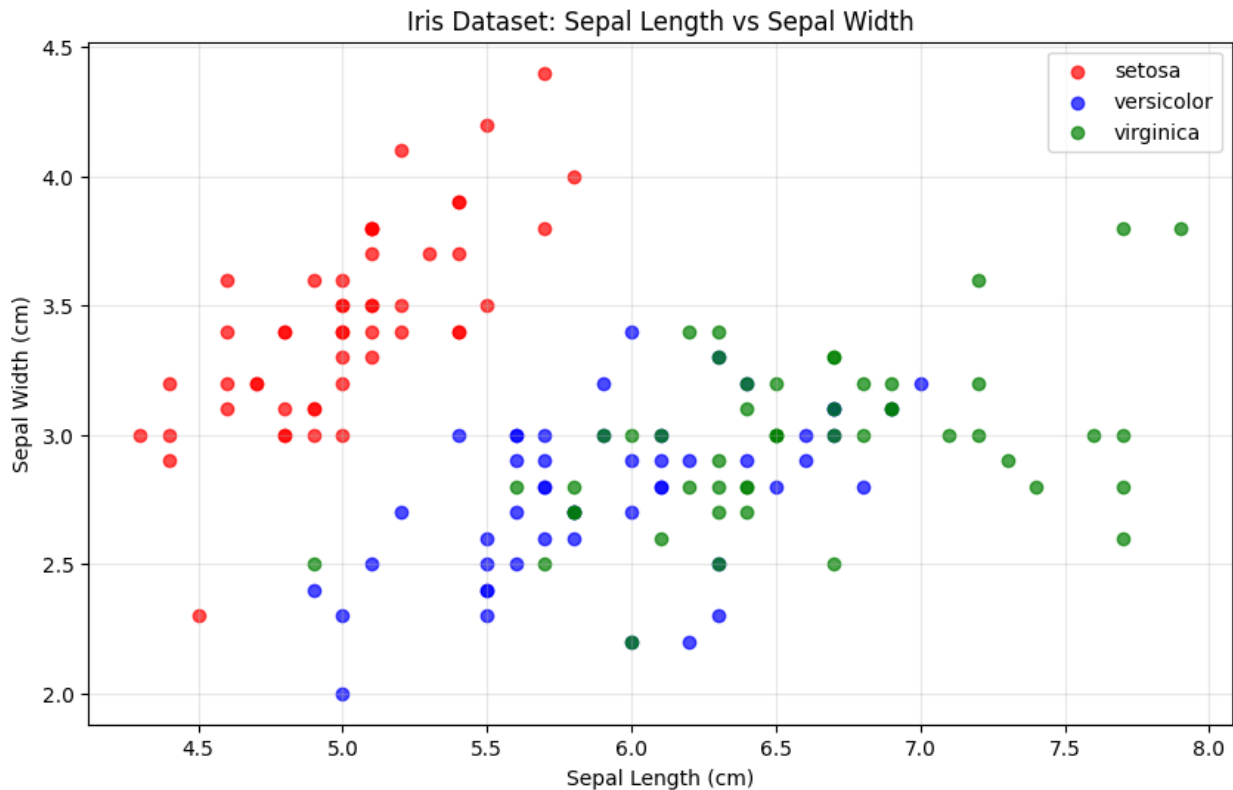
```
# Create a simple scatter plot
plt.figure(figsize=(10, 6))

# Plot sepal length vs sepal width, colored by species
species_colors = {'setosa': 'red', 'versicolor': 'blue', 'virginica':
'green'}

for species in df['species'].unique():
    species_data = df[df['species'] == species]
    plt.scatter(species_data['sepal length (cm)'],
                species_data['sepal width (cm)'],
                c=species_colors[species],
                label=species,
                alpha=0.7)

plt.xlabel('Sepal Length (cm)')
plt.ylabel('Sepal Width (cm)')
plt.title('Iris Dataset: Sepal Length vs Sepal Width')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()

print("🎉 Congratulations! You've created your first data
visualization!")
```



🎉 Congratulations! You've created your first data visualization!

Part 4: Practice with Basic Data Operations

Let's practice some basic data analysis operations that you'll use throughout the course.

```
# Basic statistical analysis
print("Basic Statistics for Iris Dataset:")
print("=" * 40)

# Calculate mean values for each species
species_means = df.groupby('species').mean()
print("\nMean values by species:")
print(species_means)

# Count samples per species
species_counts = df['species'].value_counts()
print("\nSamples per species:")
print(species_counts)
```

Basic Statistics for Iris Dataset:

=====

Mean values by species:

	sepal length (cm)	sepal width (cm)	petal length (cm)	\
--	-------------------	------------------	-------------------	---

species			
setosa	5.006	3.428	1.462
versicolor	5.936	2.770	4.260
virginica	6.588	2.974	5.552

	petal width (cm)
species	
setosa	0.246
versicolor	1.326
virginica	2.026

Samples per species:

species	
setosa	50
versicolor	50
virginica	50

Name: count, dtype: int64

Part 5: GitHub and Documentation Best Practices

Why GitHub for Machine Learning?

- **Version Control:** Track changes to your code and data
- **Collaboration:** Work with others on projects
- **Portfolio:** Showcase your work to potential employers
- **Backup:** Never lose your work

Basic GitHub Workflow:

1. **Create Repository:** A folder for your project
2. **Clone/Download:** Get the project on your computer
3. **Add Files:** Put your notebooks and data
4. **Commit:** Save a snapshot of your changes
5. **Push:** Upload changes to GitHub

For This Course:

- Create a repository named "ITAI-1371-ML-Labs"
- Upload each lab notebook as you complete it
- Include a README.md file describing your projects

Action Item: After this lab, create your GitHub account and repository.

Assessment: Tool Familiarity Check

Complete the following tasks to demonstrate your understanding of the tools:

```
# Task 1: Create a simple calculation using NumPy
# Calculate the mean and standard deviation of sepal length
```

```

sepal_lengths = df['sepal length (cm)']

# Your code here:
mean_sepal_length = np.mean(sepal_lengths)
std_sepal_length = np.std(sepal_lengths)

print(f"Mean sepal length: {mean_sepal_length:.2f} cm")
print(f"Standard deviation: {std_sepal_length:.2f} cm")

# Verification (don't modify)
assert isinstance(mean_sepal_length, (float, np.floating)), "Mean should be a number"
assert isinstance(std_sepal_length, (float, np.floating)), "Std should be a number"
print("✅ Task 1 completed successfully!")

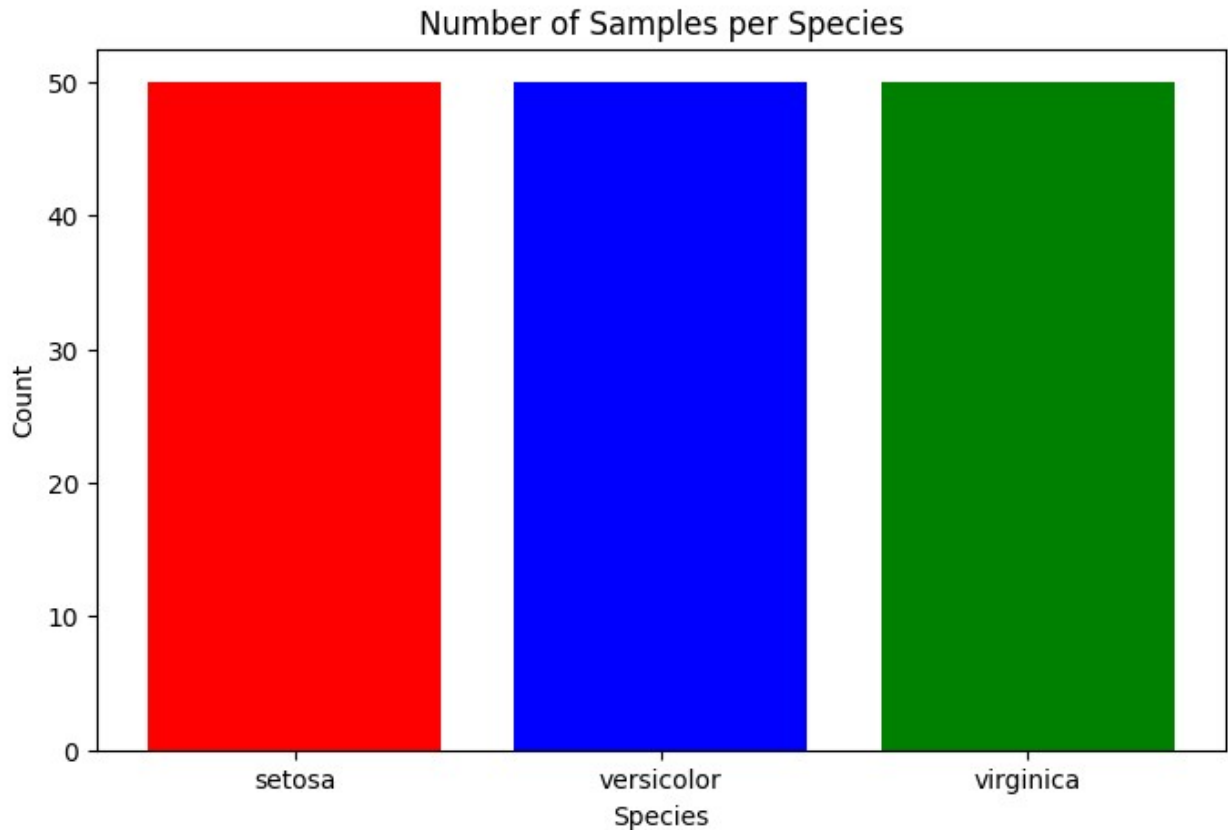
Mean sepal length: 5.84 cm
Standard deviation: 0.83 cm
✅ Task 1 completed successfully!

# Task 2: Create a simple bar chart showing species counts
species_counts = df['species'].value_counts()

plt.figure(figsize=(8, 5))
plt.bar(species_counts.index, species_counts.values, color=['red', 'blue', 'green'])
plt.title('Number of Samples per Species')
plt.xlabel('Species')
plt.ylabel('Count')
plt.show()

print(f"Species distribution: {dict(species_counts)}")
print("✅ Task 2 completed successfully!")

```



```
Species distribution: {'setosa': np.int64(50), 'versicolor':  
np.int64(50), 'virginica': np.int64(50)}  
☐ Task 2 completed successfully!
```

Your Analysis and Reflection

Instructions: Complete the analysis below by editing this markdown cell.

Edwin Marquez

My Observations About the Iris Dataset

Dataset Overview:

- Number of samples: 150
- Number of features: 4
- Number of classes: 3

Key Findings from the Visualization:

1. The scatter plot showed clear separation between setosa and the other species.
2. Versicolor and virginica overlapped slightly, indicating similar feature values.
3. Petal length and width appeared to be strong indicators for class distinction.

Questions for Further Investigation:

- How would classification accuracy change if we only used petal measurements?
- Would a non-linear model perform better than a linear one for this dataset?

Reflection: This exercise helped me understand how visualizations can reveal structure and patterns in data that aren't obvious from raw numbers. I learned how pandas and matplotlib can be used together to explore relationships between variables before modeling. Overall, the lab improved my confidence in analyzing datasets visually and preparing them for machine learning tasks.

Kendrick Robinson

My Observations About the Iris Dataset

Dataset Overview:

- Number of samples: 150
- Number of features: 4
- Number of classes: 3

Key Findings from the Visualization:

1. The scatter plot likely shows that the Iris-setosa species forms a separate, distinct cluster with a smaller sepal length and larger sepal width compared to the other two species.
2. The Iris-versicolor and Iris-virginica have more overlap in their sepal dimensions, making them harder to distinguish from each other based on sepal length and width alone.
3. The plot may show a mild negative correlation between sepal length and sepal width across the dataset, but the relationship is heavily influenced by the species.

Questions for Further Investigation:

- Would using petal length and width instead of sepal measurements create a visualization where all three species are more clearly separated?
- Which machine learning model (like K-Nearest Neighbors) would be most effective at automatically classifying the species based on these four measurements, and how accurate would it be?

Reflection: Using pandas made it easy to structure the data and perform quick analysis, like calculating group averages. Matplotlib then allowed me to visually confirm these numerical patterns, revealing clear relationships and clusters that would be difficult to see in a table of numbers.

Ali Shan

My Observations About the Iris Dataset

Dataset Overview:

- Number of samples: 150

- Number of features: 4
- Number of classes: 3

Key Findings from the Visualization:

1. The scatter plot revealed that setosa samples were clearly separated from the other species.
2. Versicolor and virginica had overlapping regions, suggesting similar measurements in certain features.
3. Petal length and petal width appeared to provide the most distinction among the three classes.

Questions for Further Investigation:

- How would the results differ if we focused only on petal-related features?
- Could clustering algorithms like K-Means effectively separate the species without labels?

Reflection: This lab helped me understand how visualization can expose important relationships in data, such as feature correlations and class separation. Using pandas and matplotlib together made it easier to analyze trends and identify which variables best distinguish each class. Overall, this exercise showed how simple visual tools can guide data preparation and model selection in machine learning.

Fernando Reyes

My Observations About the Iris Dataset

Dataset Overview:

- Number of samples: 150
- Number of features: 4
- Number of classes: 3

Key Findings from the Visualization:

1. I observed from the scatter plot that Setosa averages higher sepal widths compared to the other 2 species
2. Another observation I made was Versicolor and Virginica have similiar sepal width and length
3. Another observation I made is that, in extreme cases, Versicolor includes some of the smallest flowers, while Virginica includes some of the largest

Questions for Further Investigation:

- Can I plot more than just the sepal length and width without cluttering the data plots?
- Is it possible to make a model that runs on this data set to classify these plants given measurements with at least a 95% accuracy?

Reflection: What I learned from this exercise is how the process of how to make visualizations off of inputting databases. I also learned how to analyze the dataplots and make critical observations about the differences and similarities between the plant species.

Note: This is practice for documenting your machine learning projects professionally.

Lab Summary and Next Steps

What You've Accomplished:

- ☐ Set up your machine learning development environment
- ☐ Imported and used essential Python libraries
- ☐ Loaded and explored your first dataset
- ☐ Created your first data visualization
- ☐ Practiced professional documentation with Markdown
- ☐ Learned about GitHub for project management

Preparation for Module 3:

In the next lab, you'll:

- Learn about different types of machine learning
- Build your first simple classifier
- Understand the complete ML workflow
- Work with more complex datasets

Action Items:

1. **Create your GitHub account** and repository
2. **Upload this completed notebook** to your repository
3. **Experiment** with different visualizations using the Iris dataset
4. **Practice** Markdown formatting in a new notebook

Resources for Continued Learning:

- [Pandas Documentation](#)
- [Matplotlib Gallery](#)
- [GitHub Guides](#)
- [Jupyter Notebook Tips](#)

Great job completing Module 2! You're now equipped with the essential tools for machine learning. ☐